

Outline:

- Introduction to Kubernetes
- Cluster setup and using and creating basic Kubernetes objects – Pod, Deployment, Service and ReplicaSet.

Study environment:

- Lab materials are present on Moodle
- We will use AWS Academy resources:
 - https://www.awsacademy.com/vforcesite/LMS_Login
- In our laboratory we will use :
 - aws-cli - <https://docs.aws.amazon.com/cli/latest/userguide/install-cliv2-linux.html>
 - kubectl - <https://kubernetes.io/docs/tasks/tools/>

When you execute all commands from command line with version argument they should not exit with error

Note: If you are using desktops in Multimedia Laboratory, please select Ubuntu image (credentials inf3/inf3). Due to issues with aws-cli version please execute following command:

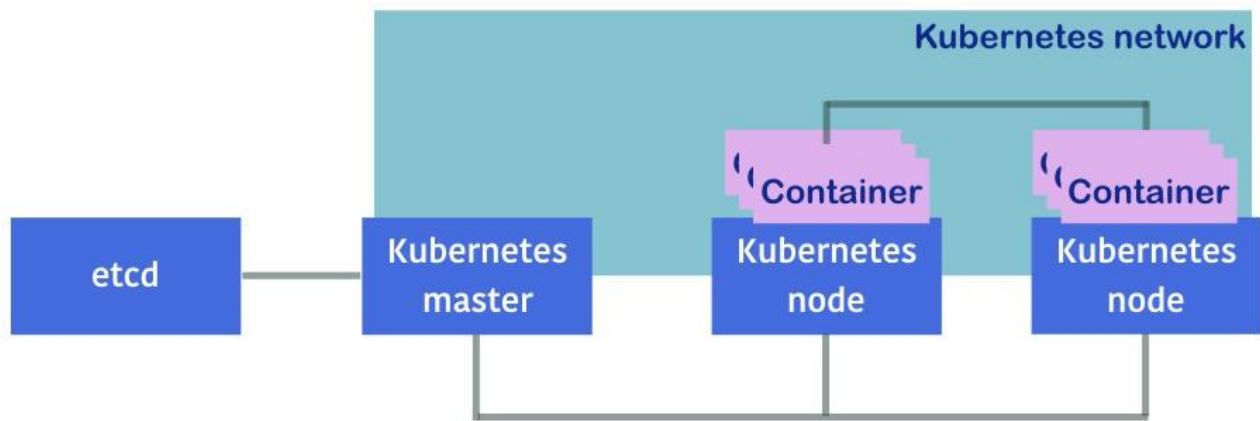
In -s /snap/bin/aws .local/bin/aws

Restart your terminal, you should use aws-cli version 2+ (command: *aws version*)

- Please combine the results of all tasks in one single text file and attach it in UPEL (at the end of class meeting).
- If you are using desktops please make sure that the time is properly set
- At the end of lab:
 - Click reset button on Vocareum labs

Kubernetes

Kubernetes is an open source orchestrator for deploying containerized applications. A Kubernetes cluster provides an orchestration API that enables applications to be defined and deployed with simple declarative syntax. Further, the Kubernetes cluster itself provides numerous online, self-healing control algorithms that repair applications in the presence of failures. Finally, the Kubernetes API exposes concepts like *Deployments*, that make it easier to perform zero-downtime updates of your software and *Service* load balancers that make it easy to spread traffic across a number of replicas of your service.



The Kubernetes API is a RESTful API based on HTTP and JSON and provided by an API server. All of the components in Kubernetes communicate through the API.

A Pod is the atomic unit of scheduling in a Kubernetes cluster. It is made up of a collection of one or more running containers. All of the containers in a Pod are guaranteed to land on the same machine in the cluster. Pods share many resources between containers, a. o.:

- network namespace,
- process and interprocess communication namespaces, so that different containers can use tools, like shared memory and signaling, to coordinate between the different processes in the Pod.

Task -1 – Prepare and Verify your environment

First, using AWS Console create cluster in Elastic Kubernetes Service (in search dialog type EKS)

Step 1
Configure cluster
Step 2
Specify networking
Step 3
Configure observability
Step 4
Select add-ons
Step 5
Configure selected add-ons settings
Step 6
Review and create

Configure cluster

Configuration options - new [Info](#)
Choose how you would like to configure the cluster.

☐ **Quick configuration (with EKS Auto Mode) - new**
Quickly create a cluster with production-grade default settings. The configuration uses EKS Auto Mode to automate infrastructure tasks like creating nodes and provisioning storage.

☒ **Custom configuration**
To change default settings prior to creation, choose this option. This configuration gives the option to use EKS Auto Mode and customize the cluster's configuration.

EKS Auto Mode - new [Info](#)
Choose if you would like to use EKS's Auto Mode.

☒ **Use EKS Auto Mode**
EKS automates routine cluster tasks for compute, storage, and networking. When a new pod can't fit onto existing nodes, EKS creates a new node. EKS combines cluster infrastructure managed by AWS with integrated Kubernetes capabilities to meet application compute needs. [View pricing](#)

► Included capabilities

Cluster configuration [Info](#)
Name
Enter a unique name for this cluster. This property cannot be changed after the cluster is created.

The cluster name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 100.
Cluster IAM role [Info](#)
Select the Cluster IAM role to allow the Kubernetes control plane to manage AWS resources on your behalf. This cannot be changed after the cluster is created. To create a new custom role, follow the instructions in the [Amazon EKS User Guide](#).
 [Create recommended role](#)

Kubernetes version settings
Kubernetes version [Info](#)
Select Kubernetes version for this cluster.

Upgrade policy [Info](#)
Choose one of the following options. You can switch the setting later while the standard support period is in effect.

☒ **Standard**
This option supports the Kubernetes version for 14 months after the release date. There is no additional cost. When standard support ends, your cluster will be auto upgraded to the next version.

☐ **Extended**
This option supports the Kubernetes version for 36 months after the release date. The extended support period has an additional hourly cost that begins after the standard support period ends. When extended support ends, your cluster will be auto upgraded to the next version.

- Step 1
Configure cluster
- Step 2
Specify networking
- Step 3
Configure observability
- Step 4
Select add-ons
- Step 5
Configure selected add-ons settings
- Step 6
Review and create

Specify networking

Networking [Info](#)

IP address family and service IP address range cannot be changed after cluster creation.

VPC [Info](#)

Select a VPC to use for your EKS cluster resources.

vpc-027fc650f81ac161a | Default [Refresh](#)

Subnets [Info](#)

Choose the subnets in your VPC where the control plane may place elastic network interfaces (ENIs) to facilitate communication with your cluster. To create a new subnet, go to the corresponding page in the [VPC console](#).

Select subnets [Refresh](#) [Clear selected subnets](#)

subnet-06edb215982ca0285 [X](#) subnet-0d8dcd442ec3ddba7 [X](#)
us-east-1a 172.31.80.0/20 us-east-1b 172.31.16.0/20

Additional security groups - optional [Info](#)

EKS automatically creates a cluster security group on cluster creation to facilitate communication between worker nodes and control plane. Optionally, choose additional security groups to apply to the EKS-managed Elastic Network Interfaces that are created in your control plane subnets. To create a new security group, go to the corresponding page in the [VPC console](#).

Select security groups [Refresh](#)

Choose cluster IP address family [Info](#)

Specify the IP address type for pods and services in your cluster.

- ☒ IPv4
- ☐ IPv6

Configure Kubernetes service IP address block [Info](#)

- ☒ Specify the range from which cluster services will receive IP addresses.

Configure remote networks to enable hybrid nodes - new [Info](#)

EKS Hybrid Nodes enables you to use on-premises and edge infrastructure as nodes in EKS clusters.

- ☒ Specify the CIDR blocks for your on-premises environments that you will use for hybrid nodes.

Cluster endpoint access [Info](#)

Configure access to the Kubernetes API server endpoint.

- ☐ Public
The cluster endpoint is accessible from outside of your VPC. Worker node traffic will leave your VPC to connect to the endpoint.
- ☒ Public and private
The cluster endpoint is accessible from outside of your VPC. Worker node traffic to the endpoint will stay within your VPC.
- ☐ Private
The cluster endpoint is only accessible through your VPC. Worker node traffic to the endpoint will stay within your VPC.

Advanced settings

[Cancel](#) [Previous](#) [Next](#)

Once it is done, add nodes to cluster. In the cluster options add nodes to Node group.

EKS > Clusters > suu-cluster

suu-cluster

[Refresh](#)
[Delete cluster](#)

Cluster info [Info](#)

Status

Kubernetes version [Info](#)

Support type

Provider

Active

1.29

Standard support until March 23, 2025

EKS

Overview

Resources

Compute

Networking

Add-ons

Access

Observability

Upgrade insights

Update history

Tags

Nodes (0) [Info](#)

< 1 >

Node name	Instance type	Node group	Created	Status
No Nodes				
This cluster does not have any Nodes, or you don't have permission to view them.				

Node groups (0) [Info](#)

Edit

Delete

Add node group

Group name	Desired size	AMI release version	Launch template	Status
No node groups				
This cluster does not have any node groups.				
Nodes that are not part of an Amazon EKS managed node group are not shown in the AWS console.				
Add node group				

- Step 1
● **Configure node group**
- Step 2
○ Set compute and scaling configuration
- Step 3
○ Specify networking
- Step 4
○ Review and create

Configure node group [info](#)

A node group is a group of EC2 instances that supply compute capacity to your Amazon EKS cluster. You can add multiple node groups to your cluster.

Node group configuration

These properties cannot be changed after the node group is created.

Name

Assign a unique name for this node group.

The node group name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 63.

Node IAM role [info](#)

Select the IAM role that will be used by the nodes. To create a new role, go to the [IAM console](#).



[Create recommended role](#)

ⓘ The selected role must not be used by a self-managed node group as this could lead to a service interruption upon managed node group deletion.

[Learn more](#)

Launch template [info](#)

These properties cannot be changed after the node group is created.

☒ Use launch template

Configure this node group using an EC2 launch template.

- Step 1
● Configure node group
- Step 2
● **Set compute and scaling configuration**
- Step 3
○ Specify networking
- Step 4
○ Review and create

Set compute and scaling configuration

Node group compute configuration

These properties cannot be changed after the node group is created.

AMI type [info](#)

Select the EKS-optimized Amazon Machine Image for nodes.

Capacity type

Select the capacity purchase option for this node group.

Instance types [info](#)

Select instance types you prefer for this node group.

t3.medium vCPU: 2 vCPUs Memory: 4 GiB Network: Up to 5 Gigabit Max ENI: 3 Max IPs: 18

Disk size

Select the size of the attached EBS volume for each node.

 GiB

Next provision the cluster. First setup AWS account on your local machine, please perform following steps:

1. You can create environment by typing *aws configure*
 - a. Add at least one data to config
2. Copy your AWS credentials from Vocareum site to local machine. Store them in *~/.aws/credentials*.
3. Create default values for session:


```
aws configure set default.region us-east-1
aws configure set default.output table
```
4. Check the environment (there should be no ec2 instances and no errors on console):


```
aws sts get-caller-identity
```

Next configure the kubectl command

```
aws eks describe-cluster --region us-east-1 --name <your cluster> --query cluster.status
aws eks --region us-east-1 update-kubeconfig --name <your cluster>
```

You should see info that the cluster is ready. Now check the status of nodes:

```
kubectl get nodes
```

Also check that all Kubernetes components are ready (here we are using *kube-system* namespace):

```
kubectl --namespace kube-system get pods
```

TASK -1 – list all nodes and components in your cluster system wide

Task 0 – Managing Containers

A Pod contains one or more containers, which run on the same host. Each Pod has its own IP address on an overlay network. All the containers inside a Pod see each other as on the same host (for example Service Mesh).

2026-update: instead of `--image=gcr.io/kuar-demo/kuard-amd64:1` use: `--image=jmutai/kuard-amd64:blue`

First create a single application with a single pod:

```
kubectl run --image=gcr.io/kuar-demo/kuard-amd64:1 kuard --dry-run=client -o yaml
```

It is generally curl file that uploads data to server, if you remove `--dry-run` section you will send request to API server.

You can make the YAML directly if the kubectl is not sufficient. It is declarative way of describing the infrastructure. You can see the complete infrastructure setup when you execute the command line:

```
kubectl get pods kuard -o yaml
```

```
kubectl get pods kuard -o wide
```

Now let's connect to the pod. First create a port forward, which is generally something like ssh-tunnel

```
kubectl port-forward kuard 8080:8080
```

Next open the browser and navigate to the application. It has some interesting features we will use. You can access the pod using following command (ash is alpine version of bash):

2026 update: `kubectl exec -ti <kuard name> -- ash`

```
kubectl exec -ti <kuard name> ash
```

Using the bash shell you can see that node address is different than the pod IP

You can also delete a pod (do not execute it yet)

```
kubectl delete pod kuard
```

In order to verify the status of the created Pod and see its details you can use following commands:

```
kubectl get pod <pod-name>
```

```
kubectl describe pod <pod-name>
```

```
kubectl logs <pod-name> <container-name>
```

However we do not want to operate on the single pods and also once pod is created it operates on that single node.

TASK 0 – copy output of the pod description

Task 1 – Managing sets of pods

Dealing with single pods at time is little complicated that is why we want to operate on sets of pods. Next create set of Pods in single deployments

```
kubectrl create deployment --image=gcr.io/kuar-demo/kuard-amd64:1 kuard --replicas=5 --dry-run=client -o yaml
```

Next create the deployment. You can check the status of deployment using
kubectrl get deployments

In addition to deployment kubectrl also creates replica sets – managed set of containers
kubectrl get rs

You can also check the definition of the ReplicaSet:
kubectrl describe rs <your replica>

Now list pods and try to delete one. What happened to the other pods ?

ReplicaSets allow the creation of many different copies of the same container image, however as applications usually are not absolutely static – it is necessary to allow a safe rollout from one version of application to another, and this is why Deployment objects were defined. A Deployment can hold pointers to multiple ReplicaSets and it can control the slow and safe migration from one ReplicaSet to another.

TASK 1 – add description of created and updated replica set

Task 2 – Services

Now, as we already know how to replicate the given application, the next step will be spreading traffic to these different replicas using a load balancer. Kubernetes defines a Service object, which represents a TCP or UDP load-balanced service. The load of request, which is sent to the Services is being distributed to Pods, based on their labels and Service's selector.

Service after its creation, is assigned a fixed virtual IP address. When traffic is sent to that IP, it is load balanced to Pods implementing the Service. Thus, components outside Kubernetes system only have to access the Kubernetes Service with an exposed network port in order to communicate with Pods, without knowing anything about containers' ports or IPs.

If you want to have access to set pods we want to expose it via service

```
kubectrl expose deployment kuard --type=LoadBalancer --port=80 --target-port=8080 --dry-run=client -o yaml
```

Once you create a service and the deployment will be created behind the internal LoadBalancer. Verify that LB is working by refreshing the page. Service description provides URL under which the LB is accessible. Because we are using the AWS EKS, Elastic Load Balancer is created on the cloud.

Task 3 – Up and running

Next open two new terminal and execute following commands in each of them:

- 1) `watch -n 0.1 kubectl get pods -o wide` (on windows `kubectl get pods -o wide --watch`)
- 2) (skip this step on windows) `export KUARD_LB=<your_lb>`
`while true ; do curl -s http://${KUARD_LB}/env/api | jq '.env.HOSTNAME'; done`

There should be no errors on each of the command lines. Next scale the deployment

```
kubectl scale deployment kuard --replicas=10
```

Observe what is happening to the pods created in the infrastructure. We can also do it manually

```
kubectl edit deployment kuard
```

There should be no errors in the requests loops.

TASK 3 – add a list of responses from different pods

Task 4 – Updating running containers

Kubernetes defines two approaches to update application on running containers in the cluster: rolling-update, which does not introduce any downtime to clients, and recreate, which terminates all Pods and recreates them.

We will use the Deployment and Service created in task 2 to see how the rolling-update strategy works. At first, check what is the default strategy, by looking at the output of *StrategyType* using:

```
kubectl create namespace deploy
kubectl create deployment -n deploy --image=gcr.io/kuar-demo/kuard-amd64:1 dep-kuard --replicas=3 --dry-run=client -o yaml
```

List all pods in the namespace (please note the naming pattern of the pods, the second element is the hash of deployment)

```
kubectl get pods -n deploy
```

Next annotate your deployment using:

```
kubectl annotate deployment/dep-kuard kubernetes.io/change-cause="replicas 2 version 1" -n deploy
```

Next describe deployment, and list Replica sets

```
kubectl describe deployment dep-kuard -n deploy
```

Looking at the output, you will see that the default strategy is rolling-update. Also, looking at the “NewReplicaSet” value you will see ReplicaSet created along with the Deployment. Let us add some changes to the deployment – you can do this using the following command and getting into

editor mode (record flag is for recording the update, please update number of replicas =5 and version of the image =2) :

```
kubectrl edit deployment dep-kuard -n deploy
```

Annotate deployment to reflect new version.

After editing the Deployment check the status of rolling-update with rollout command:

```
kubectrl rollout status deployment dep-kuard -n deploy
```

Verify the details regarding Deployment Events:

```
kubectrl describe deployment dep-kuard -n deploy
```

As you can see, a new Replica Set is created in the Deployment. How do the old and new Replica Sets behave during the scaling process? Check if your changes were applied successfully.

List all replica sets

```
kubectrl get rs -n deploy
```

List all pods in the namespace (note changed pattern in pod naming)

Make new change to the descriptor (change the replicas =7 and change version to 3)

```
kubectrl edit --record deployment dep-kuard -n deploy
```

Annotate deployment.

Rollout changes

Now, view the rollout history and check the list of revisions:

```
kubectrl rollout history deployment dep-kuard -n deploy
```

Afterwards, list the existing ReplicaSets

```
kubectrl get rs -n deploy
```

As you can see – each revision has a matched ReplicaSet and represents a stage of running an update command. Let's go back to the previous Deployment:

```
kubectrl rollout undo deployment dep-kuard --to-revision=<revision-for-rollback> -n deploy
```

List details of the pod with version of package set to history version

TASK 4 – add description of created history of deployments and details of reversed pods